A AEGIS LAB / ENGINEERING PORTFOLIO

DESKTOP FIRST · PRACTICAL ENGINEERING

AEGIS-SIM 2.0

Desktop GIS Simulation System

WPF · MVVM · .NET · Multiplayer

Army War Game Center-এর প্রকাশ্যে উল্লিখিত **Software Engineering Stack** অনুযায়ী একটি আকর্ষণীয়, বাস্তবায়নযোগ্য Portfolio Project-এর পূর্ণাঙ্গ বাংলা ব্লুপ্রিন্ট।

Windows Desktop · GIS Layers · Scenario Engine · OOP · SQL · XML · Real-Time Sync · Algorithms · Automated Tests · Replay · Explainable Analytics

DESKTOP ARCHITECTURE | MODULE SPECS | CODE SAMPLES | 14-DAY PLAN | INTERVIEW DEMO

Safety & Scope

Fully synthetic civilian disaster-response training simulator.

Edition

v2.0 · 22 October 2026

START HERE / 00

কী পরিবর্তন হলো এবং কেন

গুরুত্বপূর্ণ সতর্কতা

এটি

পোর্টফোলিও ও শেখার জন্য কাল্পনিক Civilian Emergency-Response Simulator

—সেনাবাহিনীর কোনো গোপন/বাস্তব system-এর design নয়। LinkedIn-এ উল্লিখিত প্রযুক্তি একজন কর্মীর skill list; এগুলো পুরো প্রতিষ্ঠানের বাধ্যতামূলক stack—এমন নিশ্চয়তা নেই।

তুমি যে প্রোফাইল শেয়ার করেছ, সেখানে “Multiplayer War Game Simulation System”, WinForms, .NET Framework, GIS, WPF, MVVM, XML, .NET Core, SQL এবং new features, testing, maintenance, algorithms-এর কথা আছে। সে কারণে আগের **Blazor-first** পরিকল্পনা বাদ দিয়ে এখন **WPF-first desktop application** হবে; প্রয়োজনীয় network service আলাদা ASP.NET Core-এ থাকবে।

পুরোনো পরিকল্পনা	নতুন AEGIS-SIM 2.0
Blazor web screen	WPF Windows desktop main UI
Browser SVG map	WPF GIS map; offline synthetic GeoJSON layers
General OOP service	MVVM + Commands + Data Binding + DI
Web-only real-time feed	2 WPF clients + SignalR collaboration
JSON-only scenarios	JSON + XML import/export with validation
Simple CRUD DB	SQL runs, events, snapshots, audit
AI-first presentation	Solid software first; AI optional & measured

কী দেখালে Hiring Team বেশি গুরুত্ব দিতে পারে?

প্রমাণ: দুইটি ডেস্কটপ ক্লায়েন্ট একই synthetic scenario দেখছে; Controller একটি bridge closure event দেয়; দুই স্ক্রিনে map & KPI বদলায়; event SQL-এ থাকে; replay-তে আগের state ফিরে আসে; শেষে reproducible report। এটি একটি সম্ভাব্য শক্তিশালী demo, নিয়োগের গ্যারান্টি নয়।

ENGINEERING PLAYBOOK / 01

Role-to-Project Skill Matrix

তাদের প্রকাশিত skill/কাজ	তোমার engineering deliverable	কী দেখাবে
WPF + WinForms	Modern WPF Shell; WinForms comparison mini-example	XAML desktop app; one WinForms tool optional
MVVM	Models/ViewModels/Views, observable state, commands	zero business logic in code-behind
GIS	Offline vector layer, pan/zoom, coordinates, features	load a GeoJSON layer; inspect node
.NET Framework / .NET	Modern .NET desktop + compatibility note	target framework & dependency explanation
OOP	Interfaces, immutable events, composition, invariants	class diagram & core unit tests
SQL	EF Core + SQLite/SQL Server-compatible design	migrations, indexing, data explorer
XML	Safe schema import/export, versioning	round-trip validation; malformed XML rejection
Multiplayer	Connected clients, authoritative state, versioned commands	two-window synced event demonstration
Algorithms	Deterministic ticks, capacity allocation, queue prioritization	unit tests + runtime profiling
Testing and maintenance	xUnit, logging, regression & CI	tests + change log + controlled failure

C#/.NET অগ্রাধিকার পাওয়ার তথ্য আর এই কর্মীর দক্ষতার তালিকা একসঙ্গে কাজে লাগিয়ে **পড়াশোনার priority** স্থির করা হচ্ছে। এটি অভ্যন্তরীণ requirement বা সফটওয়্যার reverse-engineering নয়।

ENGINEERING PLAYBOOK / 02

Product Brief: Synthetic Civilian Response Training

Project Name: AEGIS-SIM 2.0 — Desktop GIS Emergency Logistics Training System. **One-line pitch:** A deterministic, multiplayer-capable Windows simulation application for synthetic disaster-response exercises, featuring GIS visualization, reproducible replays, role-based control, SQL event history and optional explainable forecasts.

Three sample scenarios

S1 Flood Relief Dispatch — MVP

৩টি আশ্রয়কেন্দ্র, ১টি ডিপো, ৮টি synthetic rescue vehicles, সীমিত inventory, blocked road।

Acceptance: Start/Pause/Inject/Replay কাজ করে।

S2 Earthquake Supply Allocation — V2

কল্পিত চিকিৎসা-সামগ্রী ও খাবারের warehouse থেকে emergency shelters-এ delivery; stock ও waiting-time হিসাব।

Acceptance: A/B comparison & rule trace কাজ করে।

S3 Cyclone Communications Degradation — V3

Network delay/loss-এর কারণে data display stale হয়; reconnection এ authoritative server state ফিরে আসে।

Acceptance: offline/reconnect badge এবং event ordering ঠিক থাকে।

Personas

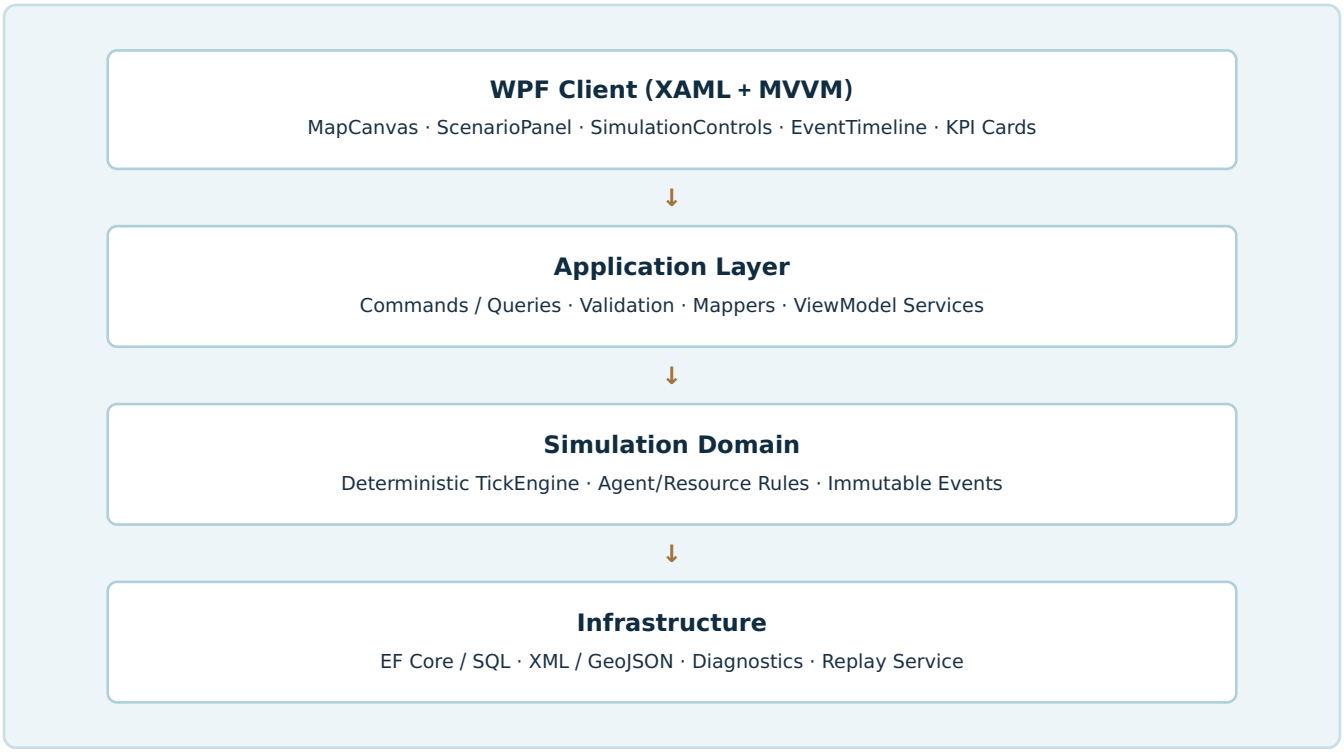
Role	Responsibility	Permission
Exercise Controller	scenario load, start/stop, insert synthetic civil incident	write commands
Observer	map, timeline, metrics, inspect details	read only
Evaluator	compare results, replay, report	read/export
Developer	log, diagnostic panel, tests	local admin

Minimum working definition

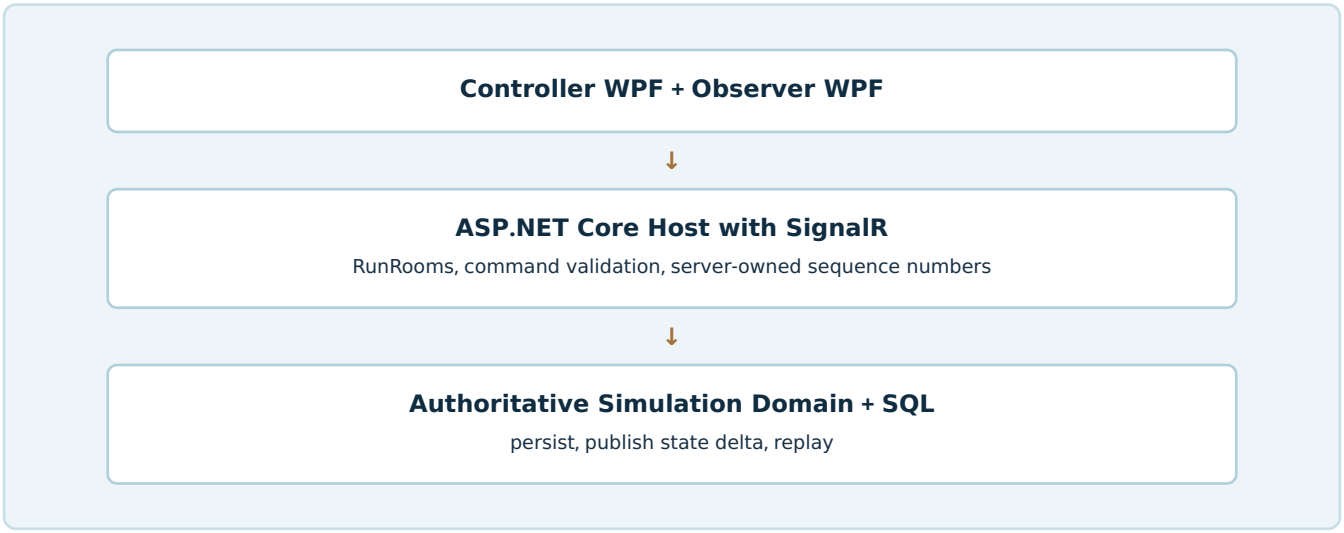
যতক্ষণ না একটি sample run-এ **Map → Event → State → SQL → Replay** সম্পূর্ণ চলবে, ততক্ষণ ML, flashy animation বা LLM যোগ করবে না।

ENGINEERING PLAYBOOK / 03

System Architecture: Desktop-First Clean Design



Network collaboration (optional, separate process)



Design decision	Reason
WPF first	Job-context desktop UI with data binding, Command, GIS control integration
Server authoritative only for multiplayer	Clients do not decide outcomes; conflicts handled centrally

Design decision	Reason
Domain has no WPF or EF dependency	Same engine in unit tests, console runner and server
Local single-player fallback	Demo runs without internet or remote APIs
Rule-based agents first	Reliability and explainability; ML layered on afterwards

> WinForms শেখার জন্য একটি ছোট legacy panel বা interoperability demo রাখবে; নতুন প্রধান desktop UI-তে WPF ব্যবহার করবে। .NET Framework বনাম আধুনিক .NET-এর migration tradeoff README-তে ব্যাখ্যা করবে।

ENGINEERING PLAYBOOK / 04

Solution and Repository Structure

REPO / FILE TREE

```
AEGIS.Sim.sln
src/
  Aegis.Desktop.Wpf/        # XAML Views, MVVM, Commands, GIS
  Aegis.Domain/             # Entities, Domain Events, Policies
  Aegis.Application/        # Use cases, DTOs, validation
  Aegis.Infrastructure/     # EF Core, XML, GeoJSON, logging
  Aegis.SyncHost/           # Optional SignalR + API host
  Aegis.Analytics/          # Metrics, comparison, optional ML
  Aegis.Tools.WinForms/     # Optional legacy UI comparison

tests/
  Aegis.Domain.Tests/
  Aegis.Application.Tests/
  Aegis.Integration.Tests/

assets/scenarios/           # synthetic JSON/XML scenarios
assets/maps/                # offline synthetic GeoJSON
scripts/                    # setup + seed + demo reset
docs/architecture/          # diagrams, ADRs, tradeoffs
.github/workflows/          # build + test + format
```

Dependency direction

WPF → Application → Domain. Infrastructure implements domain/application interfaces but Domain imports nothing from WPF, EF Core or SignalR. SyncHost depends on Application and Infrastructure. Tests may depend on any production project. This is how তুমি “all parts of software development”-এ architectural judgment দেখাবে।

Naming conventions

- `PascalCase` classes, properties, methods; `camelCase` locals; interfaces begin with `I`.
- `View` শুধু XAML display; `ViewModel` command, observable screen state; `Service` shared application behavior.
- `DomainEvent` immutable; state mutator single engine component.
- Every bug fix gets a test and Git commit with meaningful message.

ENGINEERING PLAYBOOK / 05

WPF Fundamentals: শেখা থেকে Working Window

Essential WPF topics

- XAML layouts: Grid, DockPanel, StackPanel, DataGrid, TabControl, UserControl.
- Data Binding: OneWay, TwoWay, DataContext, ItemsSource, INotifyPropertyChanged.
- Commands: ICommand, RelayCommand, AsyncRelayCommand, CanExecute and disabled states.
- Data templates, styles/resources, converters; Dispatcher/UI thread safety.
- Dependency injection, application startup lifecycle, app settings, error handling.

Working MainWindow skeleton

XAML / illustrative scaffold

```
<Window x:Class="Aegis.Desktop.Wpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        Title="AEGIS-SIM 2.0" Height="780" Width="1280">
  <Grid Margin="12">
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="3*" />
      <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <StackPanel Orientation="Horizontal" Grid.ColumnSpan="2">
      <Button Content="Start" Command="{Binding StartCommand}" />
      <Button Content="Pause" Command="{Binding PauseCommand}" />
      <TextBlock Text="{Binding TickLabel}" Margin="12,5" />
    </StackPanel>
    <ContentControl Grid.Row="1" Content="{Binding MapVm}" />
    <DataGrid Grid.Row="1" Grid.Column="1"
              ItemsSource="{Binding RecentEvents}" />
  </Grid>
</Window>
```

এই XAML structure illustrative: `MapVm` কে ContentControl-এ রেন্ডার করতে DataTemplate বা custom control register করতে হবে। সম্পূর্ণ চালু application-এর জন্য proper startup DI ও XAML namespaces লাগবে।

ENGINEERING PLAYBOOK / 06

MVVM: Real Separation, Real Testability

Responsibilities

Layer	What belongs	What must NOT belong
View	styles, grid, animations, resource references	SQL, simulation rules, raw web calls
ViewModel	observable properties, loading state, command wiring	domain state mutation logic
Model / Domain	resource, route, scenario, tick, event invariants	WPF controls and bindings
Application Service	Start/Pause/Replay use cases, transaction orchestration	visual styles or page layout

CommunityToolkit.Mvvm ViewModel

C# / partial illustrative example

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

public partial class SimulationViewModel : ObservableObject
{
    private readonly ISimulationController controller;

    [ObservableProperty] private int currentTick;
    [ObservableProperty] private bool isRunning;

    public SimulationViewModel(ISimulationController controller)
        => this.controller = controller;

    [RelayCommand(CanExecute = nameof(CanStart))]
    private async Task StartAsync()
    {
        await controller.StartAsync();
        IsRunning = true;
    }

    private bool CanStart() => !IsRunning;
}
```

> উপরের snippet concept বোঝানোর জন্য। Production-এ pause/stop পরে CanExecute notification, UI-thread dispatch, error state, cancellation এবং service lifetime সামলাতে হবে।

MVVM acceptance

- UI control না চালিয়েও Start command-এর unit test লেখা যায়।
- একই ViewModel fake data source দিয়ে test screen render করতে পারে।
- Code-behind কেবল visual concern ও framework wiring রাখে।
- Agent state পরিবর্তনের পর observable collection safe UI thread-এ update হয়।

ENGINEERING PLAYBOOK / 07

GIS and Mapping: সবচেয়ে গুরুত্বপূর্ণ Differentiator

প্রকৃত military GIS-এর দরকার নেই। নিজস্ব ছোট synthetic city-map বানাও; তাতে depot, shelter, roads, incidents, vehicles ও service zones থাকবে। **WPF Mapsui** ব্যবহার করে vector GeoJSON layer/feature visualization বা map control integrate করতে পারো। Library version অনুযায়ী API যাচাই করবে।

Map feature and layer contract

Layer	Data fields	UI behavior
Base map / area	Synthetic polygon, name, local coordinates	pan, zoom, legend
Roads	id, start, end, length, status	highlight closure (no tactical routing)
Shelters	id, capacity, currentLoad, supplies	hover/select details
Depots	id, availableInventory, serviceArea	inventory popup
Response resources	id, location, status, assignedShelter	animated location marker
Incidents	id, type, tick, areaId, severity	timeline-to-map focus

Coordinate discipline

- MVP: use synthetic planar coordinates in meters (`x,y`) for deterministic physics; label local units clearly.
- GeoJSON interoperability হলে WGS84 longitude/latitude convention ও projection differences সম্পর্কে জানবে; raw planar meters-কে সত্যিকারের GPS হিসেবে দেখাবে না।
- Zoom/Pan ও click hit-testing আলাদা test করবে; world-to-screen conversion এক জায়গায় রাখবে।
- Geometry validity: no NaN; positive extents; unknown feature IDs rejected.

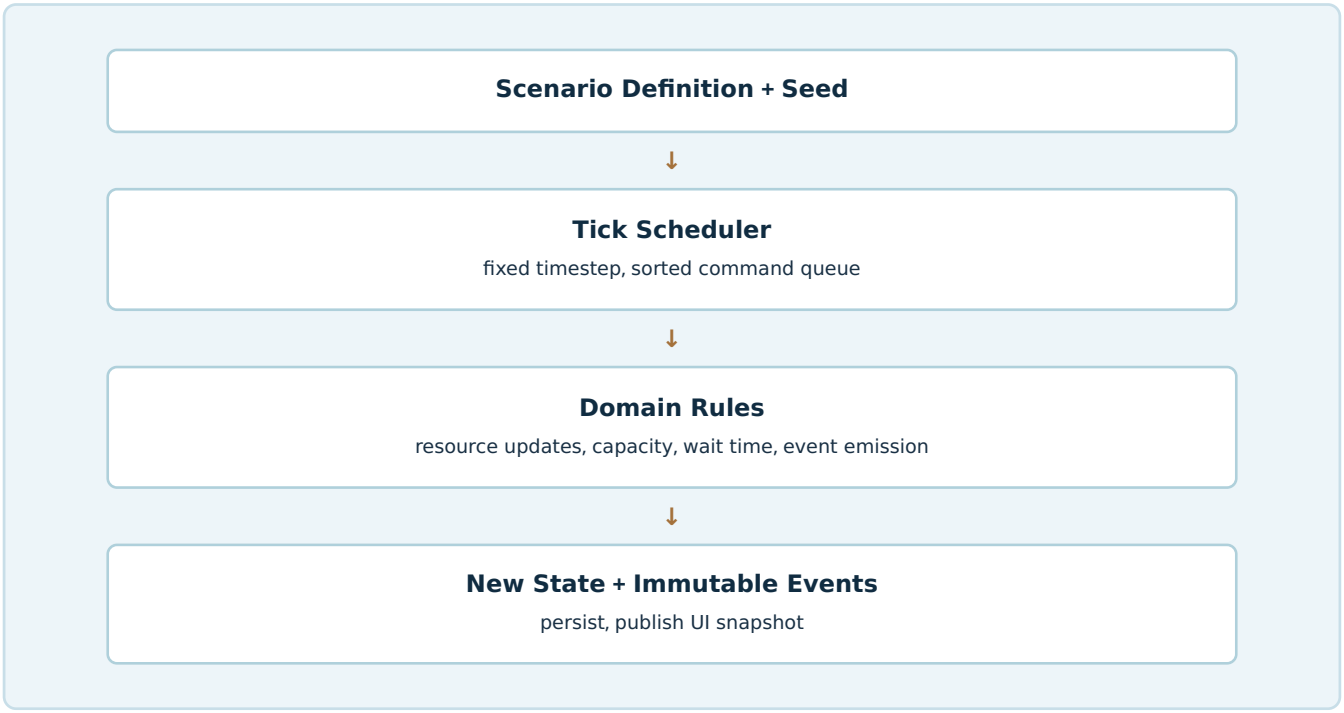
GIS milestone

একটি offline WPF window-এ ১টি map, ৬টি shelter/depot node, ১০টি road segment, ৮টি resource marker এবং event অনুযায়ী icon/status পরিবর্তন। **এই demo-টাই সবচেয়ে আগে কাজ করাও।**

ENGINEERING PLAYBOOK / 08

Simulation Engine: Core OOP and Algorithms

Simulation মানে শুধু icon animate করা নয়। এটি state transition, event ordering, constraints এবং reproducibility। UI clock ও engine clock আলাদা থাকবে।



Domain interfaces

C#

```
public record SimCommand(Guid Id, long Tick, string Type,
                         string TargetId, string Payload);
public record SimEvent(long Sequence, long Tick, string Type,
                       string EntityId, string Description);

public interface ISimulationEngine
{
    SimulationState Current { get; }
    TickResult Step(IReadOnlyList<SimCommand> commands);
    void Reset(ScenarioDefinition scenario, int seed);
}

public interface IScenarioRepository
{
    Task<ScenarioDefinition> LoadAsync(string scenarioId);
}
```

Deterministic engine contract

- Scenario + seed + version + ordered commands = same final metrics and event sequence.
- Every tick applies validated commands first, then domain rules, then emits immutable events.
- No DateTime.Now/random external I/O inside core simulation; inject time/random abstractions.

- Invalid inputs cannot produce negative inventory, duplicated resource ownership or impossible statuses.

ENGINEERING PLAYBOOK / 09

Tick Lifecycle and C# Implementation Pattern

C# / algorithm sketch

```
public TickResult Step(IReadOnlyList<SimCommand> commands)
{
    if (Current.Status != RunStatus.Running)
        throw new InvalidOperationException("Run is not active");

    long tick = Current.Tick + 1;
    foreach (var command in commands.OrderBy(c => c.Id))
        ApplyValidatedCommand(command, tick);

    AdvanceResourcesOneTick();
    UpdateCivilianSupplyQueues();
    var emitted = CollectDomainEvents();
    Current = Current with { Tick = tick };
    return new TickResult(Current, emitted);
}
```

Code scope

এটি একটি **implementation pattern**, সম্পূর্ণ compile-ready engine নয়। Domain State, constructors, invariants, repositories, and command idempotency নিজে পূরণ করবে।

State machine

From	Command	To	Result
Ready	Start	Running	first tick may advance
Running	Pause	Paused	no state advance
Paused	Resume	Running	tick continues
Running/Paused	Stop	Completed	summary locked
Any active	Reset	Ready	new run ID & seed
Completed	Step	Rejected	cannot mutate completed run

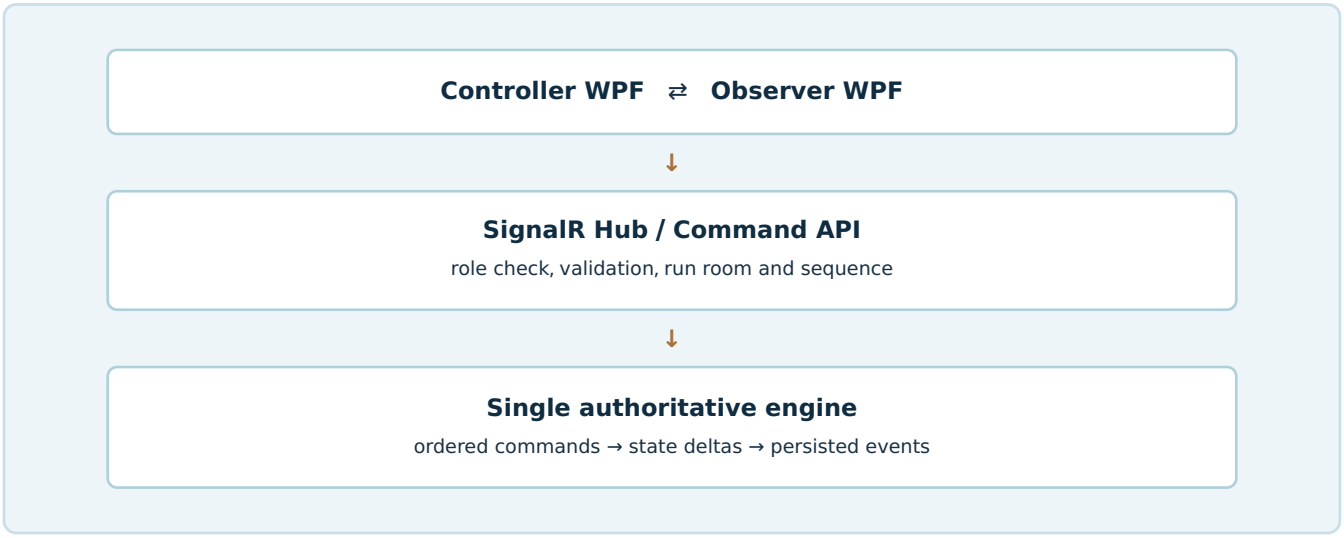
Algorithm questions you can explain

- Fixed-step vs variable-step simulation এবং speed multiplier UI refresh-কে কীভাবে প্রভাবিত করে?
- Priority queue / ordered events কোথায় কাজে লাগে?
- Capacity constraints ও FIFO queues কীভাবে maintain করো?
- Time complexity for one tick: agents N, events E, sorting E log E.
- Why deterministic replay breaks if unordered collections are enumerated differently?

ENGINEERING PLAYBOOK / 10

Real-Time Multiplayer: 2 Desktop Clients

এই অংশে কেবল একটি synthetic civilian exercise-এর shared visualization ও collaboration implement করবে। উদ্দেশ্য হলো server/client sync, versioned state, reconnect handling এবং concurrency engineering দেখানো।



Minimal message types

Message	Direction	Purpose
JoinRun(runId,lastSeenSequence)	Client → Host	subscribe and recover missing updates
SubmitCommand(commandId,runId,payload)	Controller → Host	idempotent validated action
StateDelta(runId,sequence,tick,changes)	Host → Clients	incremental display update
FullSnapshot(runId,sequence,state)	Host → Client	recovery after reconnect
RunClosed(runId,finalHash)	Host → Clients	final immutable result

Correctness requirements

- Server is authoritative. UI cannot advance core simulation alone in network mode.
- Each command has stable commandId; repeated delivery must not apply twice.
- Event sequence monotonic per run; clients reject old deltas.
- On disconnect, show “stale/connecting” label; reconnect requests snapshot or missing events.
- Do not send giant full state every frame: throttle and send small deltas.
- Observer is read-only; test unauthorized write command rejection.

ENGINEERING PLAYBOOK / 11

SQL Data Model & EF Core Persistence

Table	Core columns	Important rule
Scenarios	Id, Name, Version, ScenarioJson, CreatedAt	immutable published version
Runs	Id, ScenarioId, Seed, Status, StartAt, EngineVersion	run identity
Commands	Id, RunId, Tick, Type, Payload, UserId	unique commandId per run
Events	Id, RunId, Sequence, Tick, Type, Payload	unique (RunId, Sequence)
Snapshots	Id, RunId, Tick, StateJson, Hash	indexed RunId+Tick
Metrics	Id, RunId, Name, Value, Unit	typed metric names
Participants	Id, RunId, Role, DisplayName	authorisation context
AuditLogs	Id, RunId, Action, Timestamp, CorrelationId	append-only logs

Persistence flow

Process command → validate → simulate → write Events + Snapshots + Metrics in **one transaction** → publish StateDelta. If commit fails, never broadcast a “successful” event. For high throughput, use a consistent outbox pattern in the advanced edition.

SQL / index design

SQL / index sketch

```
CREATE UNIQUE INDEX UX_Run_Sequence
ON Events(RunId, Sequence);
CREATE INDEX IX_Snapshots_Run_Tick
ON Snapshots(RunId, Tick);
CREATE INDEX IX_Commands_Run_Tick
ON Commands(RunId, Tick);
```

> MVP development: SQLite. More complex multiuser deployment: SQL Server is a reasonable study path. Do not claim identical SQL feature support across providers; test provider-specific queries.

ENGINEERING PLAYBOOK / 12

XML, JSON, GeoJSON: Safe Scenario Interchange

স্কিল লিস্টে XML থাকায় শুধু XML parsing দেখালে চলবে না; validation, schema evolution ও round-trip handling দেখাও। এইটা maintainable enterprise programming-এর ভালো evidence।

Sample safe XML scenario

XML / synthetic scenario

```
<?xml version="1.0" encoding="utf-8"?>
<Scenario version="1" id="SYN-FLOOD-01">
  <Area widthMeters="1200" heightMeters="800" />
  <Shelter id="SH-A" capacity="50" />
  <Resource id="RV-01" supplies="20" />
  <Incident tick="35" type="RouteClosed"
            targetId="RD-03" />
</Scenario>
```

Security and validation checklist

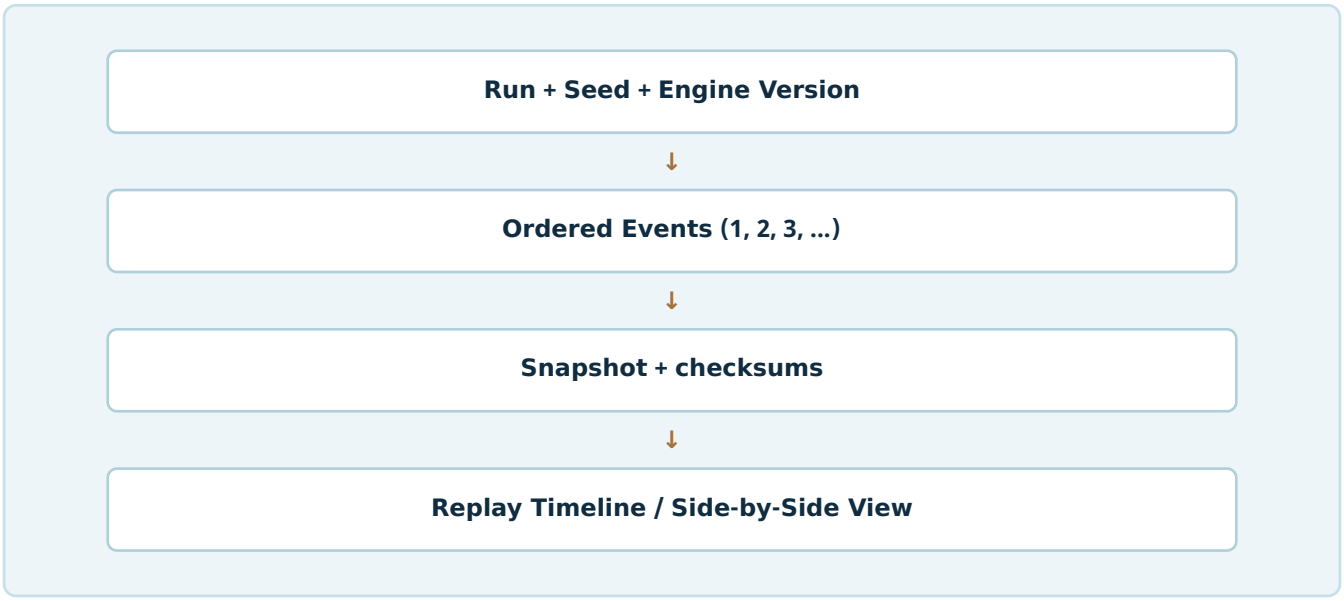
- Use `XmlReaderSettings` with DTD processing prohibited and external resolver disabled.
- Validate schemaVersion, unique IDs, units, allowed incident types, max resource count.
- Never trust external scenario file, even if locally selected; size limits & helpful error messages.
- Test XML → validated domain model → XML; preserve required fields and version.
- JSON for app default config; XML for compatibility demonstration; GeoJSON for spatial layers.

ENGINEERING PLAYBOOK / 13

Event Sourcing Lite, Snapshots and Replay

Replay should be real, not a recorded video

প্রতিটি event-এ RunId, Sequence, Tick, EventType, EntityId, Payload থাকবে। Config hash এবং EngineVersion save করবে। Snapshot প্রতি ২০ বা ৫০ ticks-এ নেওয়া যেতে পারে; interval configurable। Replay snapshot থেকে events apply করে read-only state rebuild করবে।



Test cases

Case	Expected
Same seed & same command sequence	same state hash
Load from persisted events after restart	same KPI & final hash
Duplicate command ID	ignored or rejected safely
Out-of-order delta	client requests resync
Missing snapshot	replay full log if within configured cap
Tampered event payload	checksum mismatch / fail safely

ENGINEERING PLAYBOOK / 14

Features by Priority: 18 Engineering Capabilities

F01 Desktop Shell

MVP

WPF theme, nav, keyboard actions, error/loading banners

Acceptance: 3 main pages navigable

F02 MVVM

MVP

Observable props, async commands, CanExecute

Acceptance: no business logic in code-behind

F03 Offline GIS

MVP

layer load, pan/zoom, markers, popups

Acceptance: map works offline

F04 Scenario Import

MVP

JSON + validated XML, schema versioning

Acceptance: bad files rejected

F05 Simulation Controls

MVP

Start/Pause/Resume/Step/Reset/Speed

Acceptance: transition tests pass

F06 Deterministic Core

MVP

ordered events, fixed tick, seed

Acceptance: identical final hashes

F07 Dynamic Incident

MVP

route closed, inventory change, vehicle unavailable

Acceptance: metrics change with logged event

F08 SQL Persistence

MVP

runs, events, metrics, snapshots

Acceptance: restart retains history

F09 Timeline Replay — MVP

scrubber, step to tick, snapshot seek

Acceptance: state and metric match logs

F10 Multiuser Sync — V2

host + controller + observer with SignalR

Acceptance: two desktop clients stay synced

F11 Reconnect Recovery — V2

sequence-based catch-up, stale indicator

Acceptance: simulated drop/rejoin passes

F12 Run Comparison — V2

baseline/control vs changed incident

Acceptance: side-by-side metric deltas

F13 Explainable Rule Agents — V2

resource/queue rule diagnostics

Acceptance: trace reason for each decision

F14 Analytics Dashboard — V2

resource usage, throughput, queue wait

Acceptance: calcs recompute from event data

F15 XML Export — V2

scenario & summary roundtrip

Acceptance: schema validation passed

F16 AI Forecast — ADV

synthetic completion-time regression + baseline

Acceptance: holdout MAE reported

F17 Test & Observability — ADV

xUnit, structured logging, profiling

Acceptance: CI and diagnostics evidence

F18 Installer / Releases — ADV

Windows publish, release assets, demo reset script

Acceptance: clean-machine install successful

ENGINEERING PLAYBOOK / 15

Measurement & Analytics Specification

Metric	Formula or definition	Caution
Completion rate	completed requests / generated requests	denominator must be nonzero
Mean wait time	average (serviceTick - requestTick) × tick duration	avoid mixing tick and minutes
Resource utilization	busy resource ticks / available resource ticks	document unavailable time
Average delivery latency	deliveryTick - dispatchTick	completed deliveries only
Scenario delta	metric in modified run - baseline metric	same seed / same other parameters
Run consistency	snapshot replay final hash == original	hash algorithm fixed and documented

Comparison experiment

Run A: baseline scenario seed 20261022. **Run B:** same seed + closure of synthetic road RD-03 at tick 35. Compute result differences; repeat multiple seeds if you want a meaningful mean. Never claim a single synthetic run statistically proves superiority.

Charts to include

- Time-series: completed deliveries versus tick.
- Side-by-side bars: average waiting, completion ratio, utilization.
- Event histogram: types by interval.
- Per resource activity timeline; tooltip shows event provenance.

ENGINEERING PLAYBOOK / 16

Optional AI: Useful, Measurable, Safe

AI হবে **একটি ছোট measurable module**, মূল highlight নয়। Synthetic past-run features দিয়ে “remaining completion time” অনুমান করা যেতে পারে। Model choices: simple linear regression baseline, tree regression, optional ML.NET trainer. ML তখনই রাখবে যখন baseline-এর সঙ্গে objective evaluation আছে।

AI pipeline



Explainability UI

- List real input features and their values at prediction time.
- Show model vs baseline MAE; flag unsupported out-of-distribution cases.
- For tree feature importance, document approximation or omit explainability score.
- AI-generated prose must cite actual run IDs and metrics; template report is offline fallback.

> **AI credibility rule**
>
> No predictive capability should be described as “production-ready” based only on synthetic training. It demonstrates integration and evaluation discipline, not real-world emergency-management efficacy.

ENGINEERING PLAYBOOK / 17

Windows Desktop UX: Screen-by-Screen Specification

Screen	Key elements	One must-pass interaction
Home	recent runs, quick scenario launch, health status	open saved run
Scenario Editor	synthetic map layers, forms, JSON/XML validator	save valid scenario
Live GIS Run	map, markers, step controls, KPIs, live events	inject incident & see update
Participants	controller & observer roles, connection status	observer is read-only
Replay	timeline scrubber, tick state, event description	seek and match state hash
Compare	run A/B and delta charts	see same seed warning
Report	scenario info, methods, charts, limitation tags	save PDF/CSV
Diagnostics	logs, engine version, DB path, memory/tick rate	export diagnostics

Professional polish checklist

- App-wide typographic hierarchy, light/dark readable modes, coherent symbol legend.
- Loading, empty, disconnected and validation-error states designed intentionally.
- Use keyboard shortcuts Ctrl+O import, Space pause/resume (when focus safe), R reset.
- Separate live simulation metrics from AI forecast visually.
- Do not exaggerate data as actual geography: prominent “SYNTHETIC DATA / TRAINING ONLY” label.

ENGINEERING PLAYBOOK / 18

Testing Matrix: What Makes It Production-Minded

Test level	Examples	Evidence
Unit / Domain	fixed seed determinism; negative inventory; invalid transition	xUnit green tests
MVVM	StartCommand disabled while running; errors surfaced	ViewModel tests
GIS	valid layer parsed, missing feature handled, viewport transform	unit/screenshot
Integration / SQL	schema migration, transaction rollback, reload replay	SQLite temporary DB
Sync	duplicate command, old sequence, lost connection, observer denied	host + 2 clients
Performance	1000 ticks time, UI memory, event throughput	benchmark measurements
Packaging	fresh Windows account install, offline map, clean exit	screen recording

12 concrete tests to implement

- SameSeedSameResult
- DifferentSeedNeverCausesInvalidState
- CannotStartTwice
- CannotAdvanceCompletedRun
- InventoryNeverNegative
- RunSequenceMonotonic
- UnknownEntityRejected
- XmlDtdRejected
- XmlRoundTripValid
- ReplayMatchesOriginalHash
- ObserverCannotSubmitCommand
- ReconnectReceivesMissingEvents

Quality bar

একটি “world-class” portfolio project মানে feature count নয়: small, robust architecture; reproducible behavior; documentation; demo stability; tests; disciplined claims.

ENGINEERING PLAYBOOK / 19

14-Day Fast Build Roadmap: Priority and Output

Day	Work	Daily deliverable
1	C# refresh: records/interfaces/async; solution setup	solution builds, 2 unit tests
2	Domain models, command/event schema, invariants	testable state transitions
3	Deterministic tick engine and 2 scenario rules	50 ticks run without errors
4	WPF layout, dependency injection, MVVM	Start/Pause via commands
5	Map layers, markers, pan/zoom	offline synthetic GIS screen
6	SQL schema, migration, event storage	save/reopen one run
7	Replay timeline + state checksum	persisted replay verified
8	XML/JSON import, validation, reset script	round-trip and malformed test
9	KPI cards, event detail, scenario injection	full solo MVP demo
10	SignalR host + two desktop clients	remote observer receives updates
11	Sync recovery, roles and logging	reconnect & authorization test
12	Run comparisons and report export	A/B PDF/CSV sample
13	Regression tests, profiling, README	CI with measured outputs
14	Fix demo, record video, prepare Q&A	portfolio-ready stable release

> ১৪ দিন
> **high-intensity target, guarantee নয়**
> । WPF নতুন হলে solo MVP (Days 1–9) যথেষ্ট ভালো লক্ষ্য; AI optional পরে। বাস্তবে deadline অনুযায়ী skill priority সাজাবে।

ENGINEERING PLAYBOOK / 20

Learning Order and Free Official Resources

Order	Learn / build	Resource
01	C# language, LINQ, generics, async/await	learn.microsoft.com/dotnet/csharp
02	WPF XAML, Binding, Commands, UI thread	learn.microsoft.com/dotnet/desktop/wpf
03	CommunityToolkit.Mvvm	learn.microsoft.com/dotnet/communitytoolkit/mvvm
04	Mapsui WPF map layers	mapsui.com
05	EF Core, SQLite, migrations	learn.microsoft.com/ef/core
06	ASP.NET Core SignalR	learn.microsoft.com/aspnet/core/signalr
07	System.Xml and safe XML parsing	learn.microsoft.com/dotnet/api/system.xml
08	xUnit and Microsoft testing docs	learn.microsoft.com/dotnet/core/testing
09	ML.NET (only after MVP)	learn.microsoft.com/dotnet/machine-learning

Beginner pacing (if switching from Python/JS)

দিন ১–২ console C# + unit tests → দিন ৩–৪ WPF data binding and ViewModel → দিন ৫ GIS map → তারপর SQL and collaboration। যে feature নিজে explain করতে পারো না, demo-তে impressive হলেও technical interview-তে দুর্বলতা হয়ে যাবে।

ENGINEERING PLAYBOOK / 21

AI Coding Prompts: Implement Responsibly

Prompt A / Solution Skeleton

> Create a C# .NET Windows WPF solution with Domain, Application, Infrastructure and Desktop projects. Use CommunityToolkit.Mvvm, built-in DI and xUnit. Show file-by-file code, compile commands and tests. Do not introduce GIS or networking yet.

Prompt B / Deterministic Domain

> Implement a fixed-tick synthetic disaster supply simulation with immutable events, explicit seed, no wall-clock dependencies and 10 invariants. Write xUnit tests before UI. No military-specific mechanics.

Prompt C / WPF and MVVM

> Build MainWindow and 3 ViewModels: Scenario, RunWorkspace, Replay. Use XAML data binding, RelayCommands and cancellation. Zero business rules in code-behind. Explain dependencies and threading.

Prompt D / GIS Adapter

> Create an IMapAdapter interface for a WPF offline synthetic-map display. Define sample GeoJSON layers and map markers. Keep map library types outside Domain. Include map loading error state.

Prompt E / SQL and XML

> Add EF Core SQLite persistence for Runs, Commands, Events and Snapshots. Add safe versioned XML scenario import with DTD disabled. Include migrations and integration tests.

Prompt F / Multiplayer

> Add optional ASP.NET Core SignalR run rooms for two WPF clients. One authoritative host, command deduplication, versioned state deltas, reconnect snapshot and observer read-only tests.

Prompt G / Audit and Portfolio

> Review this solution for race conditions, replay inconsistency, UI-thread issues and unvalidated inputs. Produce failing tests first, then minimal fixes. Draft a truthful README with feature status and limitations.

Rules when using AI-generated code

- Ask for one compileable increment at a time; run tests before next prompt.
- Do not paste unreviewed giant files; inspect code boundaries and migrations.
- Ask AI to describe what it assumes, what may not compile, what to test.
- Avoid undocumented package versions; pin them and include lock/config state.
- Never fabricate benchmark numbers, screenshots or features in your portfolio.

ENGINEERING PLAYBOOK / 22

Interview Demo: 3 Minutes + 8 Deep Technical Questions

Time	Live action	Engineering signal
0:00–0:25	Launch Windows app; choose synthetic Flood-01; show GIS layers	WPF + offline GIS
0:25–0:55	Start engine, move resources, show KPI & event sequence	OOP + engine
0:55–1:25	Close road RD-03; observe changed wait and persisted log	rules + SQL
1:25–1:55	Open second desktop client; show synced update	multiplayer sync
1:55–2:25	Disconnect/reconnect observer; show recovery	reliability
2:25–3:00	Replay run and display summary + tests	reproducibility & QA

Technical follow-up answers to prepare

- Why WPF over WinForms? When legacy WinForms still makes sense?
- Explain DataContext, Binding Mode, ObservableProperty and RelayCommand.
- How do you prevent blocking the UI thread?
- Why separate GIS adapter from the domain model?
- How do you guarantee deterministic replay?
- What if two clients send conflicting commands simultaneously?
- How does an index on RunId+Sequence improve replay queries?
- What is your hardest bug, reproducer, regression test, and fix?

ENGINEERING PLAYBOOK / 23

CV / GitHub / Demo Submission Packaging

Headline for CV

> AEGIS-SIM 2.0 — Desktop GIS Simulation Platform (C#, WPF, MVVM, EF Core, SQL, XML, SignalR)
>
> Built a synthetic disaster-response training simulator featuring deterministic state transitions, offline map visualization, event persistence, replay and optional synchronized multi-client exercise views.

Repository checklist

- README: what works, what is planned, setup, demo GIF/video, tech stack.
- docs/architecture: C4-style boxes, MVVM diagrams, data flow, key tradeoffs.
- docs/tests: test list and reproducible commands; performance measurements with hardware noted.
- Synthetic data license/provenance and explanation of no real operational data.
- Release notes and screenshots that match real current code.
- Git history showing incremental design, bug fixes and testing.

A strong 40-second explanation

“আমি C# WPF দিয়ে একটি desktop GIS training simulator তৈরি করেছি। এর core engine UI থেকে আলাদা এবং same seed দিয়ে reproducible run তৈরি করে। SQL-এ event history রাখি, MVVM দিয়ে UI testable রেখেছি, এবং two-client mode-এ authoritative host ব্যবহার করেছি। এই demo-তে একটি synthetic incident দিয়ে outcome change, replay এবং test দেখাচ্ছি।”

ENGINEERING PLAYBOOK / 24

Risk Register and Decision Rules

Risk	Impact	Mitigation
Unfamiliar WPF XAML	High	minimal 1-view shell first; defer visual polish
GIS library breaking changes	Medium	pin version, wrap IMapAdapter
UI freezes	High	keep engine off UI thread; throttle dispatch
Multiplayer hard to debug	High	finish single-player first; logging + sequence IDs
Replay mismatch	High	same seed/hash golden tests since Day 2
SQL data corruption	High	transaction consistency, migrations & backup
XML security	Medium	disable DTD, validate, constrain size
Overambitious AI	High	defer until measured baseline and stable engine
Deadline pressure	High	scope freeze and choose tested MVP over advanced demo
Disclosure/credibility	High	mark synthetic, no internal Army claims
Unknown reviewer preference	Medium	demonstrate basic programming + documentation
No internet during demo	High	offline map and local host all core features

ENGINEERING PLAYBOOK / 25

Definition of Done: Gate Before Calling It Complete

- ☐ Fresh Windows PC can install/run the WPF application.
- ☐ Scenario JSON loads and invalid XML fails safely.
- ☐ 1 offline GIS map + nodes + roads + resource icons renders.
- ☐ Start/Pause/Resume/Step/Reset transitions behave correctly.
- ☐ Same seed and commands produce identical final hash.
- ☐ One synthetic incident changes simulator state and metrics.
- ☐ Event log persists in SQLite after app restart.
- ☐ Replay reconstructs state and matches stored hash.
- ☐ Two WPF clients show same authoritative tick / sequence.
- ☐ Observer role cannot mutate run state.
- ☐ Reconnect produces no duplicates or invented events.
- ☐ 12 key tests pass and test commands are documented.
- ☐ README identifies limitations honestly.
- ☐ Video shows functional features without fake screens.
- ☐ Portfolio makes no claim to reproduce restricted military software.

Recommended build order

WPF + MVVM → GIS → DETERMINISTIC ENGINE → SQL → REPLAY → MULTIPLAYER → ANALYTICS → OPTIONAL AI

Correct, working engineering beats a giant unfinished feature list.

APPENDIX / 26

Reference Links and Verification Notes

Microsoft WPF documentation
https://learn.microsoft.com/en-us/dotnet/desktop/wpf/

Microsoft MVVM Toolkit
https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/

C# language guide
https://learn.microsoft.com/en-us/dotnet/csharp/

.NET SDK downloads
https://dotnet.microsoft.com/en-us/download

WinForms documentation
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/

Entity Framework Core
https://learn.microsoft.com/en-us/ef/core/

ASP.NET Core SignalR
https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction

XML secure parsing (.NET API)
https://learn.microsoft.com/en-us/dotnet/api/system.xml.xmlreadersettings

Mapsui mapping library
https://mapsui.com/

ML.NET documentation
https://learn.microsoft.com/en-us/dotnet/machine-learning/

xUnit
https://xunit.net/

GitHub Actions .NET
https://docs.github.com/en/actions/how-tos/writing-workflows/building-and-testing/building-and-testing-net

LinkedIn skill list is user-provided contextual information, not verified internal specifications. Feature list, map and algorithms in this document are synthetic educational choices. Product names and tool APIs evolve, so validate current package versions before implementation.